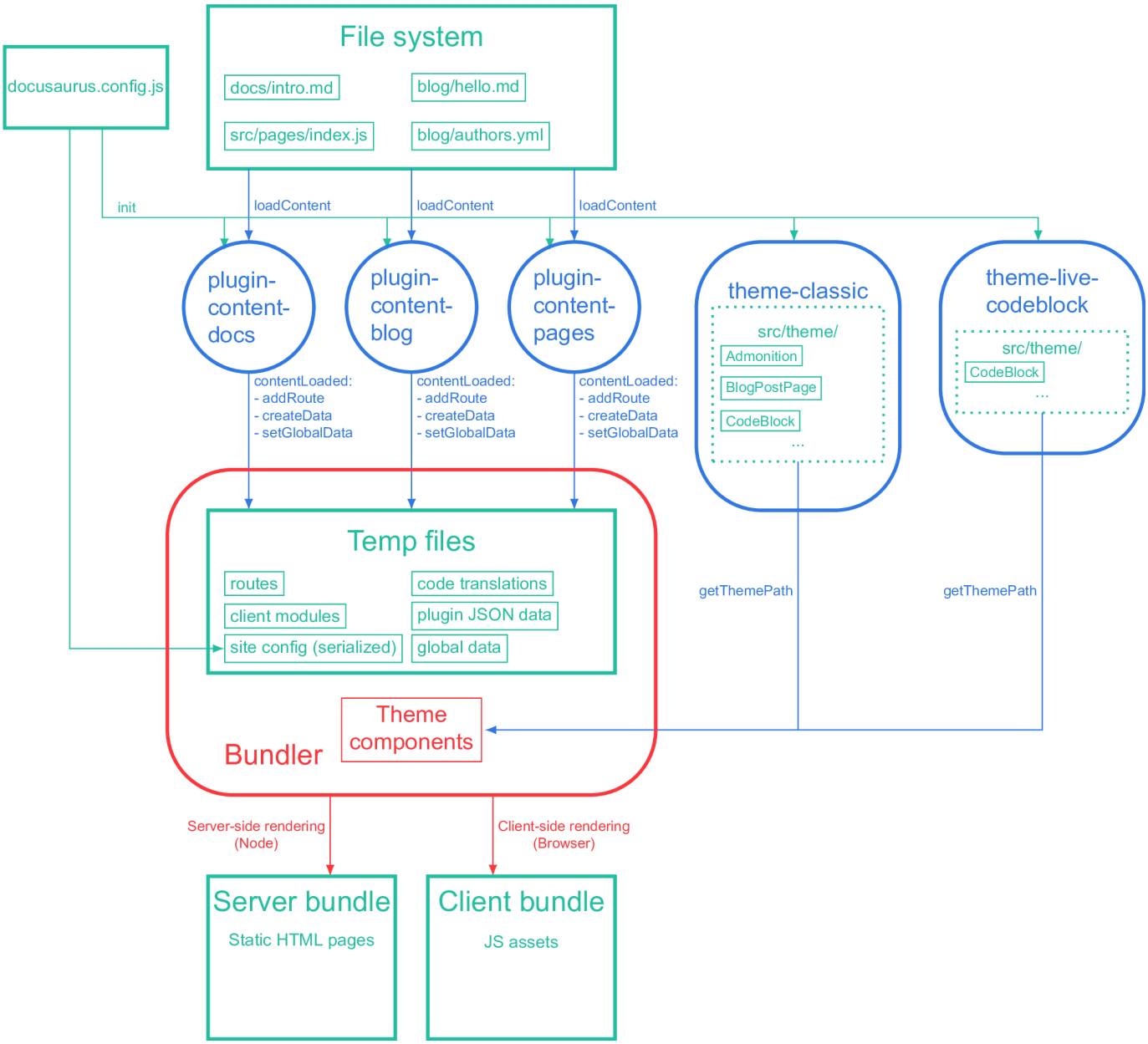
Version: 3.10.1

Architecture



This diagram shows how Docusaurus works to build your app. Plugins each collect their content and emit JSON data; themes provide layout components which receive the JSON data as route modules. The bundler bundles all the components and emits a server bundle and a client bundle.

Although you (either plugin authors or site creators) are writing JavaScript all the time, bear in mind that the JS is actually run in different environments:

- All plugin lifecycle methods are run in Node. Therefore, until we support ES Modules in our codebase, plugin source code must be provided as ES modules that can be imported, or

CommonJS that can be `require`'d.

- The theme code is built with Webpack. They can be provided as ESM—following React conventions.

Plugin code and theme code never directly import each other: they only communicate through protocols (in our case, through JSON temp files and calls to `addRoute`). A useful mental model is to imagine that the plugins are not written in JavaScript, but in another language like Rust. The only way to interact with plugins for the user is through `docusaurus.config.js`, which itself is run in Node (hence you can use `require` and pass callbacks as plugin options).

During bundling, the config file itself is serialized and bundled, allowing the theme to access config options like `themeConfig` or `baseUrl` through `useDocusaurusContext()`. However, the `siteConfig` object only contains **serializable values** (values that are preserved after `JSON.stringify()`). Functions, regexes, etc. would be lost on the client side. The `themeConfig` is designed to be entirely serializable.

*Last updated on **Apr 30, 2026** by **Sébastien Lorber***